

HW2-swgl-1

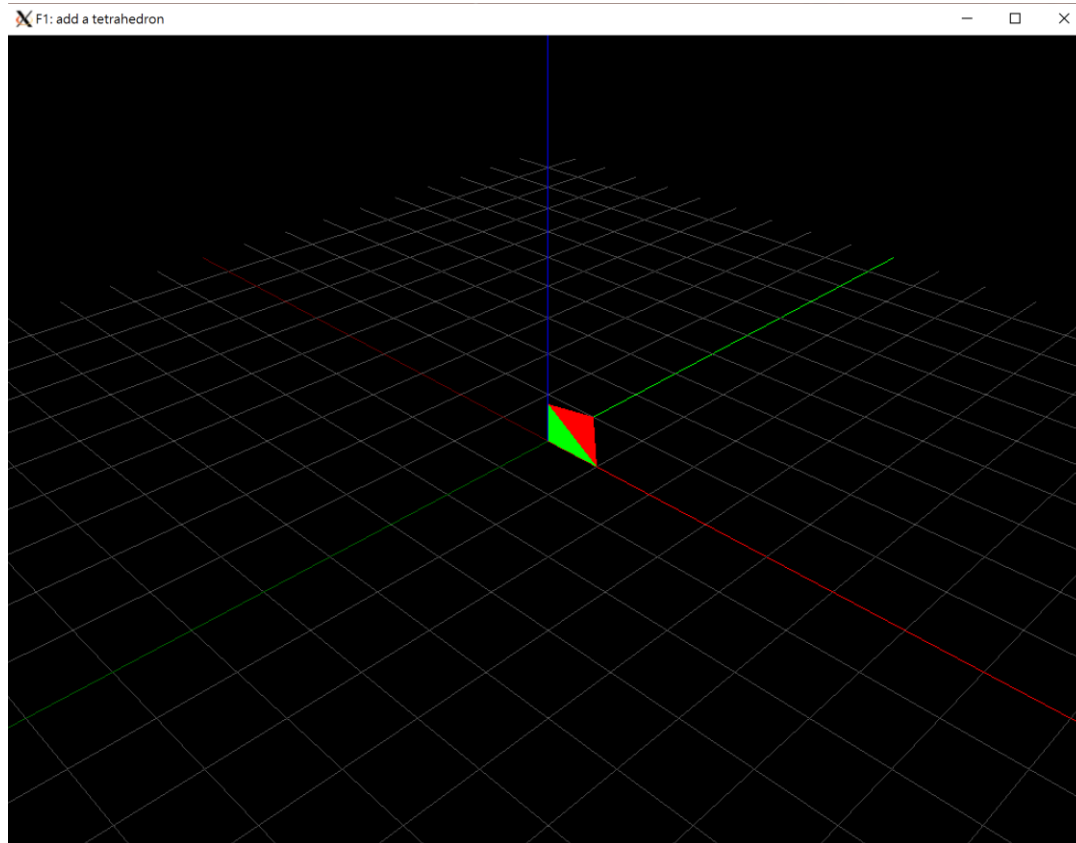
transformation & projection

CG26

NCCU

Goal

- Implement the **transformation** & **projection** functions in main.cpp



Display()

```
void Display(...) {  
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);  
  
    glMatrixMode(GL_PROJECTION);  
    glLoadIdentity();  
    gluPerspective(60, 1, 0.1, 50);  
    glMatrixMode(GL_MODELVIEW);  
    glLoadIdentity();  
    gluLookAt(10 * cos(theta), -10 * sin(theta), 10, 0, 0, 0, 0, 0, 1);  
  
    DrawGrid();  
    Draw_Tetrahedron();  
  
    glfwSwapBuffers(window);  
}
```

```
void Draw_Tetrahedron() {  
    glBegin(GL_TRIANGLES);  
  
        swTriangle(vec3(1, 0, 0), tetrahedron_verts[0], tetrahedron_verts  
        swTriangle(vec3(0, 0, 1), tetrahedron_verts[3], tetrahedron_verts  
        swTriangle(vec3(0, 1, 0), tetrahedron_verts[2], tetrahedron_verts  
        swTriangle(vec3(1, 1, 0), tetrahedron_verts[1], tetrahedron_verts  
  
    glEnd();  
}
```

Core

```
void swTriangle(vec3 color, vec3 in_v1, vec3 in_v2, vec3 in_v3, mat4x4 Modelmatrix){  
    vec4 v1(in_v1.x, in_v1.y, in_v1.z, 1); ...  
  
    v1 = Modelmatrix * v1; ...  
  
    glColor3f(color.r, color.g, color.b);  
    glVertex3f(v1.x, v1.y, v1.z);  
    glVertex3f(v2.x, v2.y, v2.z);  
    glVertex3f(v3.x, v3.y, v3.z);  
}
```

UI

```
void KeyCallback(GLFWwindow* window, int key, int scancode, int action, int mods) {  
    switch (key) {  
        case GLFW_KEY_W:  
            transformMat = swTranslate(0, 1, 0) * transformMat;  
            break;  
    }
```



Matrix vector mutilation

$$\begin{bmatrix} \\ \\ \\ \end{bmatrix} = \begin{bmatrix} 0 & 4 & 8 & 12 \\ 1 & 5 & 9 & 13 \\ 2 & 6 & 10 & 14 \\ 3 & 7 & 11 & 15 \end{bmatrix} \times \begin{bmatrix} 0 & 4 & 8 & 12 \\ 1 & 5 & 9 & 13 \\ 2 & 6 & 10 & 14 \\ 3 & 7 & 11 & 15 \end{bmatrix}$$

$$\begin{bmatrix} \\ \\ \\ \end{bmatrix} = \begin{bmatrix} 0 & 4 & 8 & 12 \\ 1 & 5 & 9 & 13 \\ 2 & 6 & 10 & 14 \\ 3 & 7 & 11 & 15 \end{bmatrix} \times \begin{bmatrix} 0 \\ 1 \\ 2 \\ 3 \end{bmatrix}$$

Step 0: mat4x4

```
mat4x4 swTranslate(float x, float y, float z)
{
    mat4x4 Translate = mat4x4(1);
    Translate[3][0] = x;
    //todo: y z

    return Translate;
}
```


Step 1: Transformation

- Use the matrix and vector lib in glm
- Implement following functions
 - mat4x4 **swTranslate**(float x, float y, float z)
 - mat4x4 **swRotateX**(float angle)
 - mat4x4 **swRotateY**(float angle)
 - mat4x4 **swRotateZ**(float angle)
 - mat4x4 **swScale**(float x, float y, float z)

Resource

- OpenGL 2.1 Reference Pages
 - <https://www.khronos.org/registry/OpenGL-Refpages/gl2.1/>
- Glm: OpenGL Mathematics
 - <https://glm.g-truc.net>

Display()

```
void Display(...) {  
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);  
  
    glMatrixMode(GL_PROJECTION);  
    glLoadIdentity();  
    gluPerspective(60, 1, 0.1, 50);  
    glMatrixMode(GL_MODELVIEW);  
    glLoadIdentity();  
    gluLookAt(10 * cos(theta), -10 * sin(theta), 10, 0, 0, 0, 0, 0, 1);  
  
    DrawGrid();  
  
    Draw_Tetrahedron();  
  
    glfwSwapBuffers(window);  
}
```

Step 2: viewing

```
const bool STEP2 = true;
```

- In display()

```
//step 2
```

```
if (STEP2 == true) {
```

```
    glMatrixMode(GL_MODELVIEW);
```

```
    glLoadIdentity();
```

```
    //ViewMat = mat4x4(1);
```

```
    ViewMat = swLookAt(10 * cos(theta), -10 * sin(theta), 1
```

```
}
```

//step2: remove glLookAt, compute view matrix

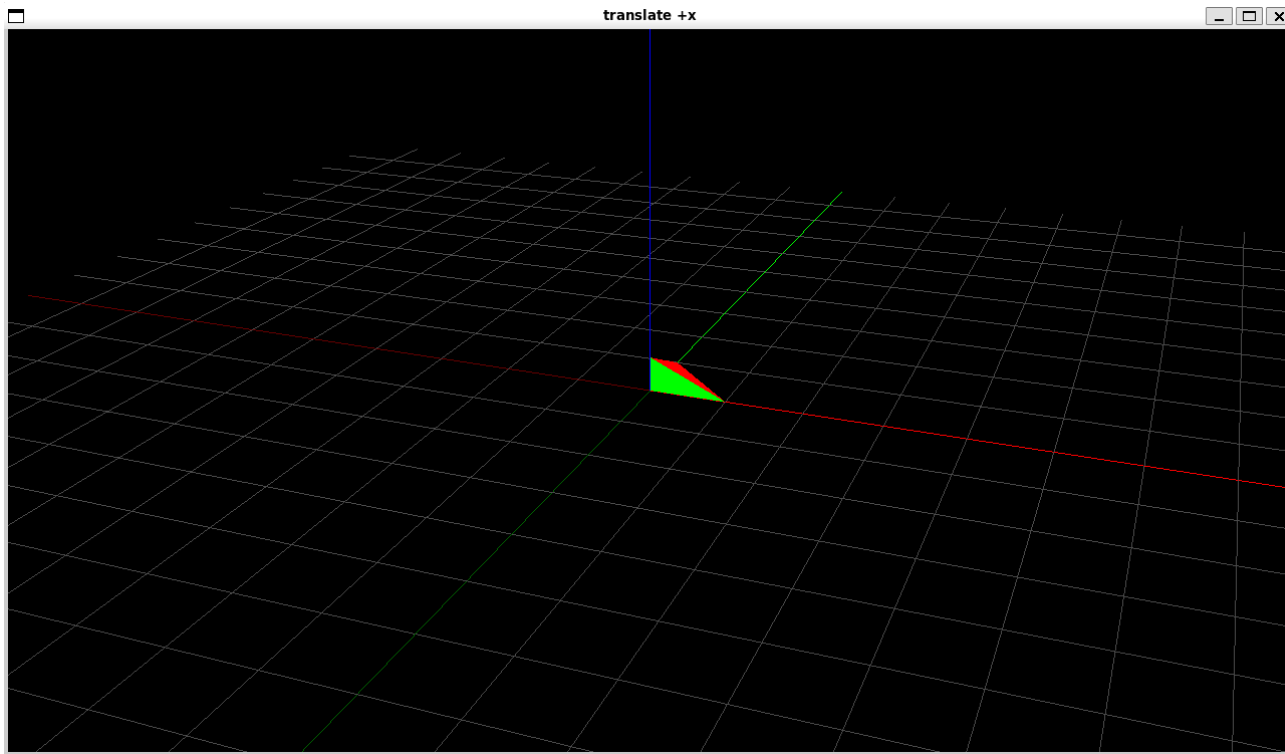
//v1 = **View*** Modelmatrix * v1;

//

//

check

- Press 9 or 0 to test swLookat()



Step 3: Projection

```
const bool STEP3 = true;
```

- In display()

```
    if (STEP3 == true) {  
        glMatrixMode(GL_PROJECTION);  
        glLoadIdentity();  
glOrtho(0, winWidth, 0, winHeight, -2.0, 2.0);  
        //ProjectionMat = mat4x4(1);  
        ProjectionMat = swPerspective(60, 1, 0.1, 50);  
    }
```

//step3: remove glProjection, compute project matrix

v1 = Projection * View * Modelmatrix * v1;

//...

//

perspective division

// ...

Question

- local /global transformation?
 - Thinking the order of matrix multiple

Extra

- Add other models (cube, sphere, loading obj file)
- Hierarchy (push/pop Current Transformation matrix)
- Load/save scene
- Triangle Rasterization

NOTE

The Life of a vertex

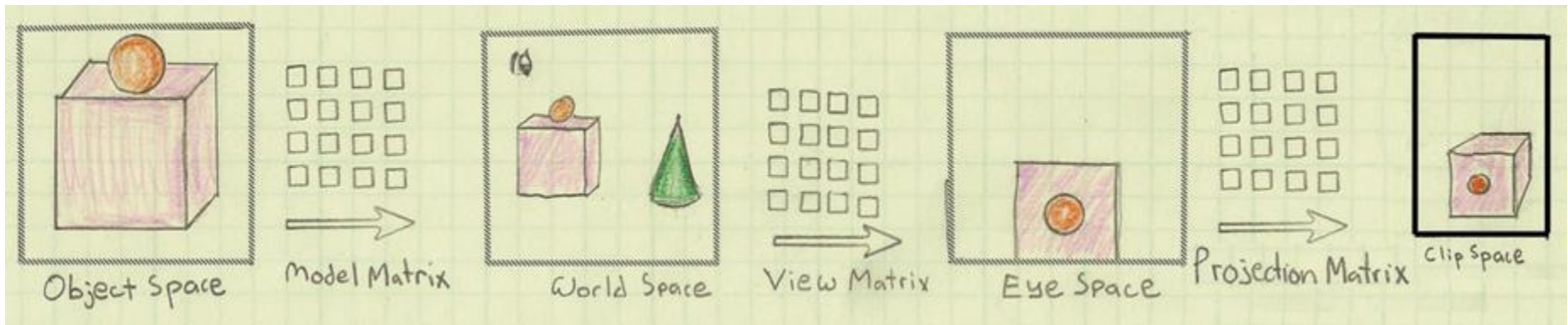


Image by Philp Rideout

OpenGL spec 1.3 [[pdf](#)]

If a vertex in object coordinates is given by $\begin{pmatrix} x_o \\ y_o \\ z_o \\ w_o \end{pmatrix}$ and the model-view matrix

The specified matrix replaces the current matrix with the one pointed to. **MultMatrix** takes the same type argument as **LoadMatrix**, but multiplies the current matrix by the one pointed to and replaces the current matrix with the product. If C is the current matrix and M is the matrix pointed to by **MultMatrix**'s argument, then the resulting current matrix, C' , is

$$C' = C \cdot M.$$

If a vertex in object coordinates is given by $\begin{pmatrix} x_o \\ y_o \\ z_o \\ w_o \end{pmatrix}$ and the model-view matrix

is M , then the vertex's eye coordinates are found as

$$\begin{pmatrix} x_e \\ y_e \\ z_e \\ w_e \end{pmatrix} = M \begin{pmatrix} x_o \\ y_o \\ z_o \\ w_o \end{pmatrix}.$$

Similarly, if P is the projection matrix, then the vertex's clip coordinates are

$$\begin{pmatrix} x_c \\ y_c \\ z_c \\ w_c \end{pmatrix} = P \begin{pmatrix} x_e \\ y_e \\ z_e \\ w_e \end{pmatrix}.$$

The vertex's normalized device coordinates are then

$$\begin{pmatrix} x_d \\ y_d \\ z_d \end{pmatrix} = \begin{pmatrix} x_c/w_c \\ y_c/w_c \\ z_c/w_c \end{pmatrix}.$$